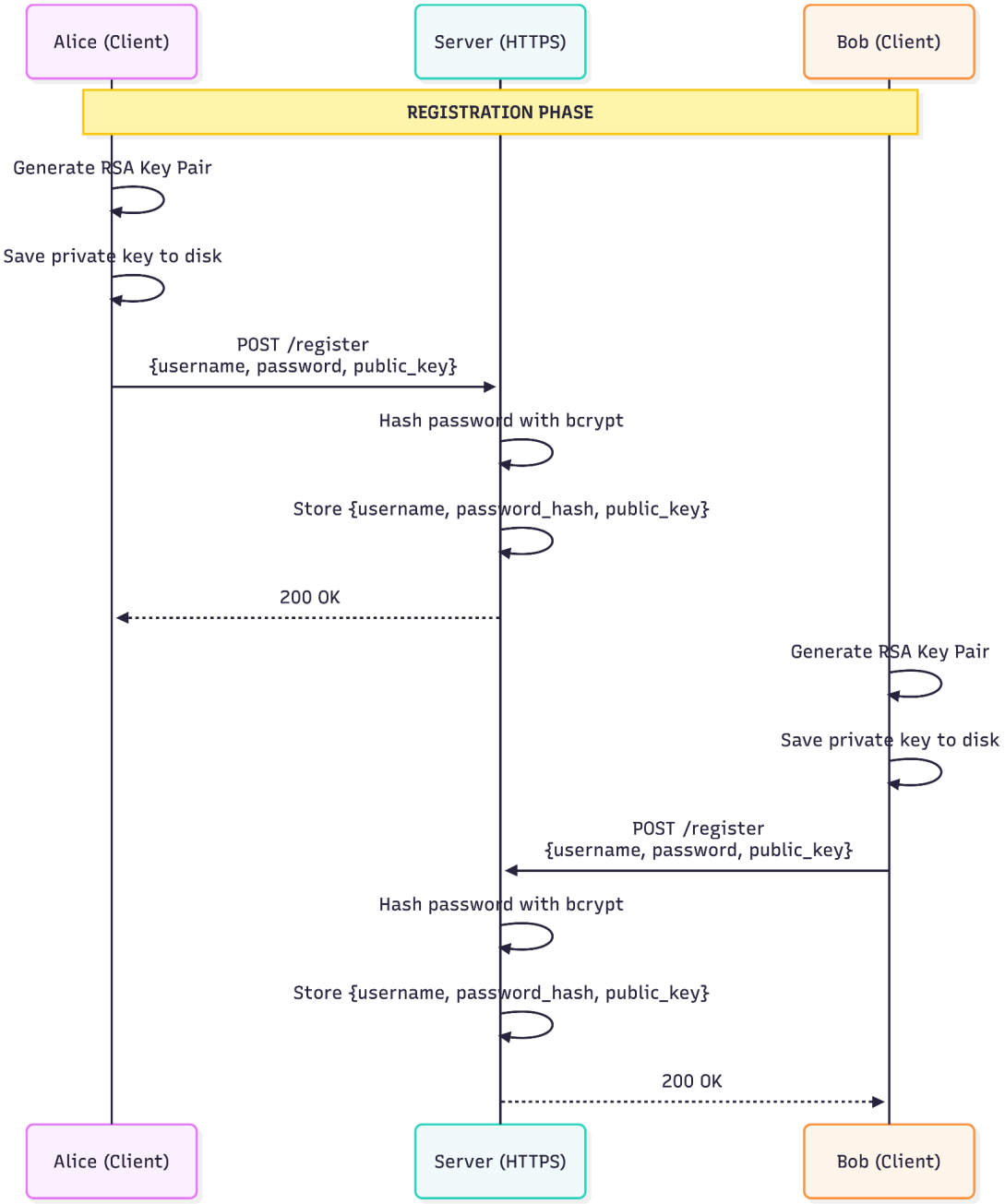
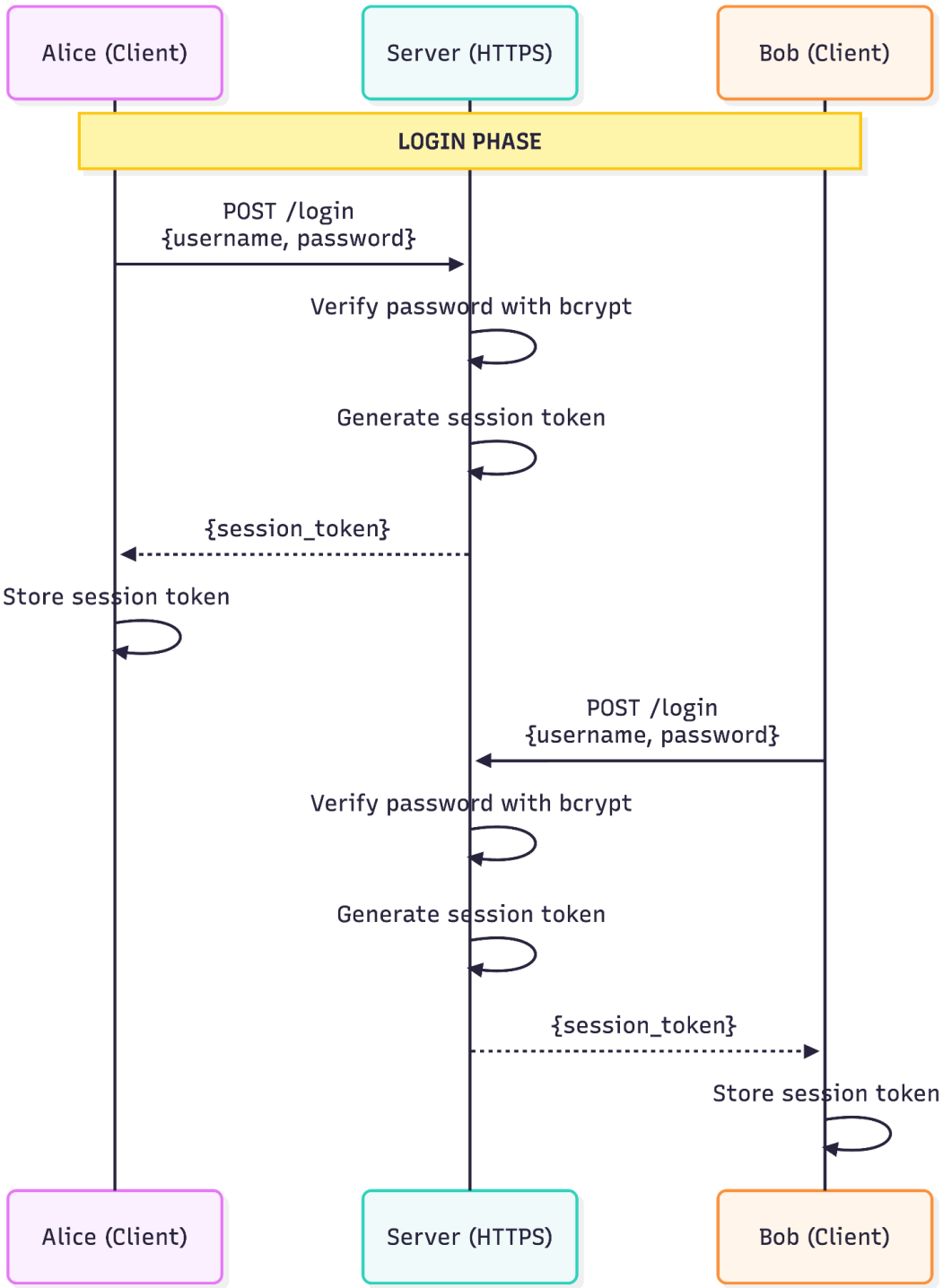
System Flow diagram :

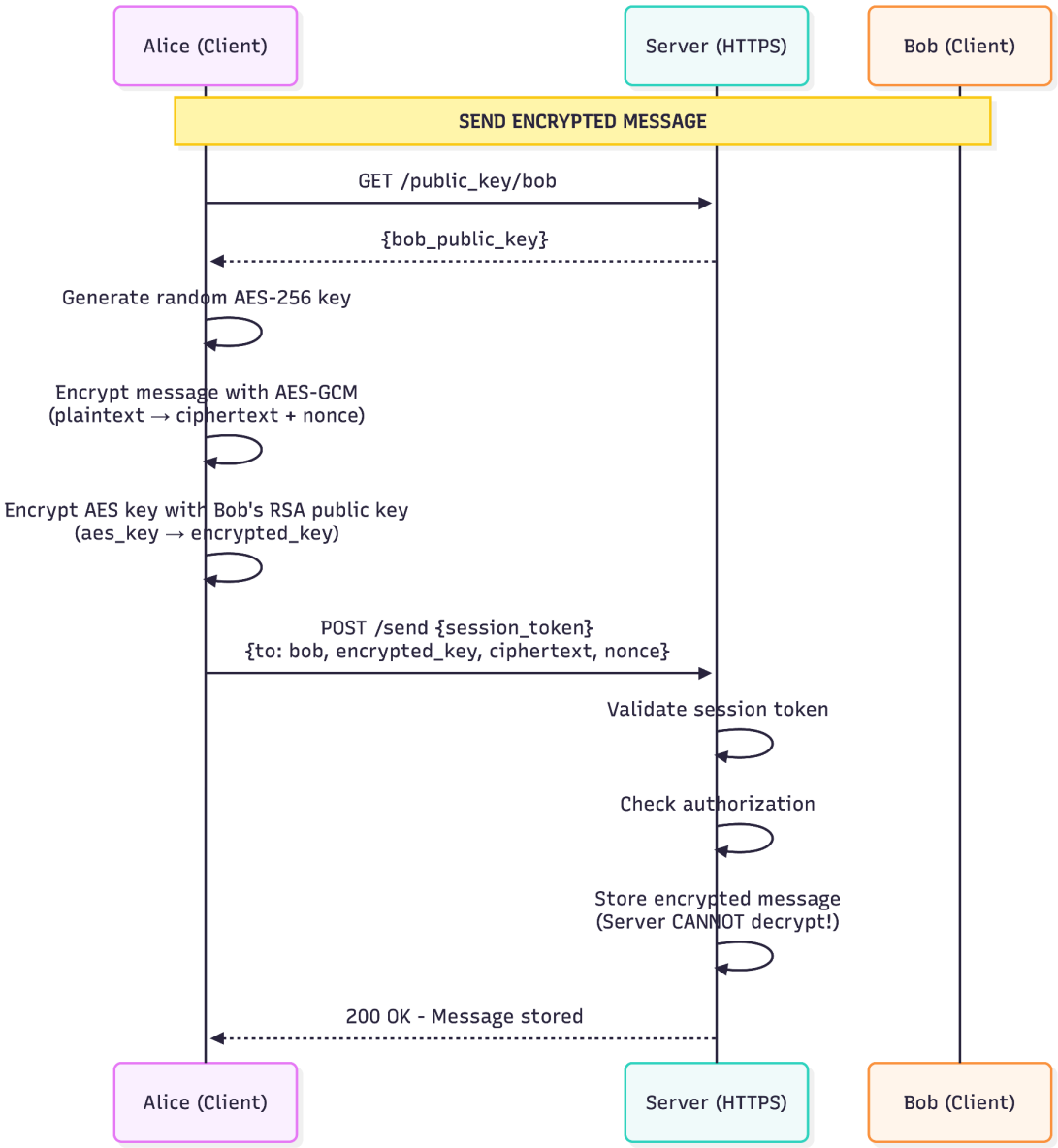
1. Registration Phase



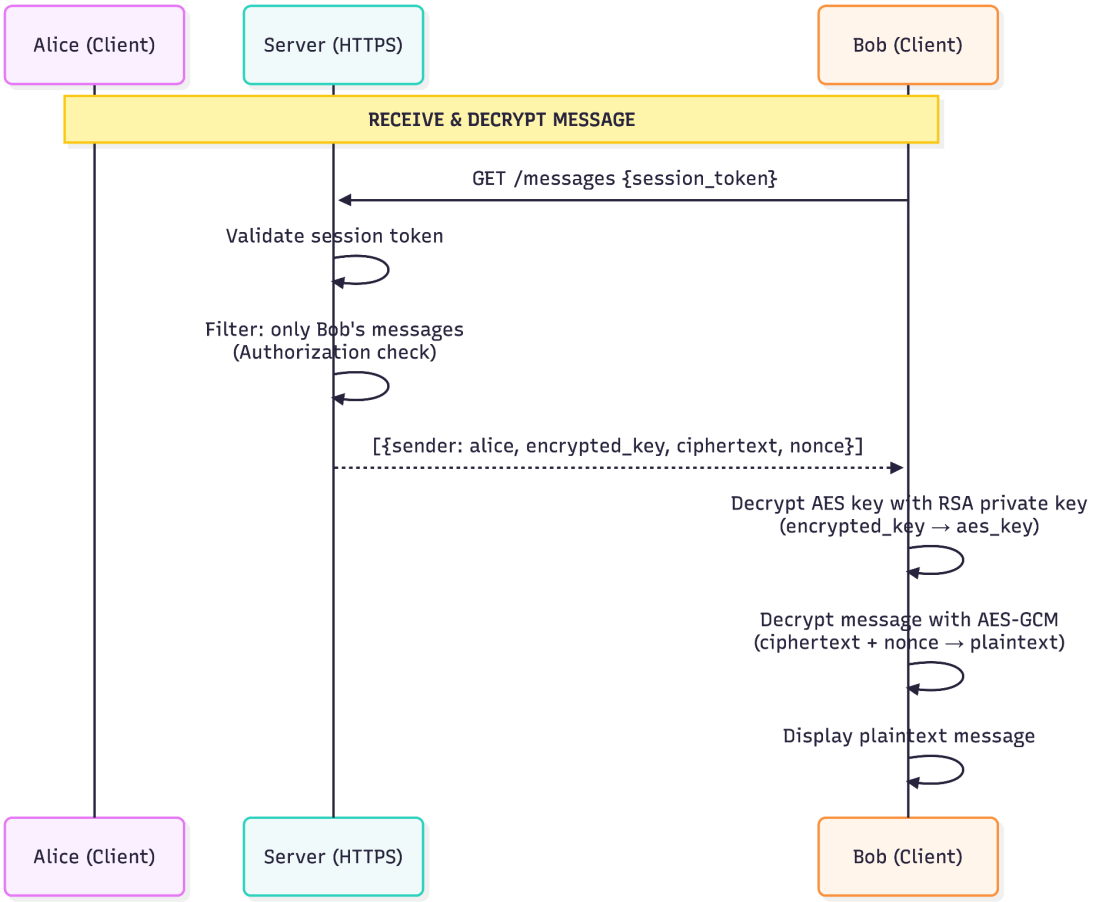
2. Login phase



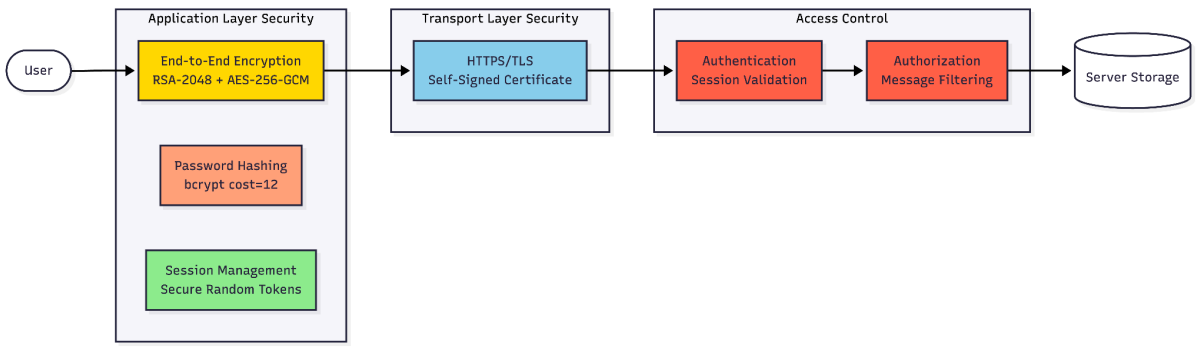
3. Sending encrypted message phase



4. Decryption phase :



Security Layes diagram :



Security Justifications

1. Why Hybrid Encryption

We use hybrid encryption because RSA and AES each solve different problems.

RSA is good for key exchange but it is slow and has size limits. You cannot encrypt large messages with RSA because it can only handle 245 bytes at a time with 2048-bit keys.

AES-GCM is very fast and can encrypt large messages but the problem is how do you share the AES key securely over the network. If you send the key in plaintext anyone can intercept it.

Hybrid encryption solves both problems. We use RSA to encrypt the AES key which is only 32 bytes. Then we use AES-GCM to encrypt the actual message which can be any size. This gives us both security and performance.

RSA-2048 is secure until 2030 according to NIST standards and AES-256 is military grade encryption that would take billions of years to brute force.

2. Why Passwords Are Hashed Not Encrypted

Passwords must be hashed not encrypted because hashing is one way. You cannot reverse a hash to get the original password.

If we encrypted passwords we would need to store the encryption key somewhere on the server. If an attacker compromises the server they get both the encrypted passwords and the key. Then they can decrypt all passwords.

With bcrypt hashing even if the server is compromised the attacker only gets hashes. They cannot reverse the hash to get passwords. They would have to brute force each password individually which is very expensive with bcrypt.

Bcrypt also adds a random salt to each password. This means two users with the same password will have different hashes. This prevents rainbow table attacks where attackers use pre-computed hash tables.

The cost factor of 12 in bcrypt means each password takes about 0.3 seconds to hash. This is fine for login but makes brute forcing millions of passwords impractical.

3. Why Sessions Are Needed

Sessions prevent sending passwords with every request.

Without sessions you would need HTTP Basic Auth where the password is sent with every single request. If any request is intercepted the attacker gets your password. Even over HTTPS this increases the attack surface.

With session tokens the password is only sent once during login. After that we use a random session token for all requests. If a token is stolen the damage is limited.

Session tokens are generated using secrets.token_urlsafe which uses cryptographically secure random generation. They are not sequential or predictable like simple counters would be.

Tokens expire after 24 hours so even if stolen they have limited lifetime. They can also be revoked immediately if suspicious activity is detected.

This is similar to how websites keep you logged in. You login once and then use cookies for subsequent requests instead of typing your password every time.

4. Why Authorization Is Enforced Server Side

Authorization must be server side because clients cannot be trusted.

If the client filtered its own messages a malicious user could simply modify the client code. They could remove the filter and request all messages from the server. Then they would see everyone's messages.

The server is the only trusted component. Only the server can enforce who can access what data.

Our server checks if the recipient matches the logged in user before returning messages. This happens in code that the client cannot modify.

```
user_messages = [msg for msg in messages if msg['recipient'] == username]
```

This line ensures Alice can only see messages sent to Alice. Bob cannot trick the system into giving him Alice's messages.

This is a fundamental security principle. Never trust the client. Always validate and enforce rules on the server.

5. PKI Trust Assumptions

Our PKI model makes several trust assumptions.

First we assume the server stores public keys correctly. When Alice registers her public key we trust the server doesn't replace it with a malicious key. If the server stored a fake public key controlled by an attacker then the attacker could decrypt messages meant for Alice.

Secondly we use self-signed TLS certificates. This means we trust the certificate on first connection. An attacker could perform man in the middle on the very first connection and substitute their own certificate. After that all traffic goes through the attacker. This is called the TOFU problem which means Trust On First Use.

In production systems this is solved with Certificate Authorities. The CA vouches that a public key belongs to a specific domain. Browsers come with trusted CA certificates pre-installed.

Thirdly we assume private keys stay private. If someone steals your private key file they can decrypt all past and future messages sent to you. We offer optional password protection for private keys but many users skip this for convenience.

Fourth we assume the cryptographic algorithms are secure. We trust that RSA-2048 and AES-256 have no mathematical weaknesses. If someone discovers a flaw in these algorithms the entire system breaks.

6. Attacks Still Possible

This system does not prevent all attacks. Here are the main limitations.

1. Metadata Leakage
   The server sees who sends messages to whom and when. Even though message content is encrypted the server knows Alice talks to Bob every day at 3pm. This metadata can reveal patterns and relationships.

2. No Rate Limiting
   Nothing stops an attacker from trying millions of passwords. If a user has a weak password like 123456 it can be brute forced quickly. Production systems should add rate limiting and account lockouts after failed attempts.

3. Man in the Middle on First Connection
   Self-signed certificates are vulnerable to MITM on first use. An attacker on the network during first connection can substitute their certificate. After that they can decrypt all traffic. This requires the attacker to be present during first connection.

4. No Forward Secrecy
   If your private key is compromised all past messages can be decrypted. Someone who recorded your encrypted messages six months ago can decrypt them now if they steal your key today. Systems like Signal use forward secrecy where old messages cannot be decrypted even if current keys are stolen.

5. Denial of Service
   An attacker can flood the server with requests and make it unavailable. Nothing in our system prevents this. Production systems need DDoS protection and request throttling.

6. Server Compromise
   If the server is compromised the attacker can see all future messages. They just add logging to see encrypted payloads as they arrive. They still cannot decrypt them but they can modify the server code to store malicious public keys for new users.

7. Weak Password Choice
   We hash passwords properly but we cannot prevent users from choosing password123. There is no password complexity requirement or strength checker.

8. No Multi-Factor Authentication
   A stolen password gives full account access. Production systems should require a second factor like SMS code or authenticator app.